

Queries con Relaciones

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

-  Realizar JOINS entre tablas relacionadas
-  Entender lazy loading vs eager loading
-  Optimizar queries para evitar el problema N+1
-  Filtrar y ordenar por campos de relaciones

Contenido

1. El Problema N+1

Cuando accedes a relaciones sin cuidado, puedes generar muchas queries:

```
# ❌ PROBLEMA N+1
def list_posts_with_authors(db: Session) -> list[Post]:
    posts = db.execute(select(Post)).scalars().all() # 1 query

    for post in posts:
        print(f"{post.title} by {post.author.name}") # N queries!

    return posts
```

Si tienes 100 posts, esto genera **101 queries** (1 + 100).

2. Lazy Loading vs Eager Loading

Lazy Loading (Por defecto)

```
# Las relaciones se cargan cuando las accedes
post = db.get(Post, 1) # Query 1: SELECT * FROM posts WHERE id=1
print(post.author.name) # Query 2: SELECT * FROM authors WHERE id=...
print(post.tags) # Query 3: SELECT * FROM tags JOIN post_tags...
```

-  Eficiente si no siempre necesitas las relaciones
-  Genera N+1 si iteras sobre muchos objetos

Eager Loading

Carga las relaciones **junto con** el objeto principal:

```
from sqlalchemy.orm import joinedload, selectinload

# JOIN en una sola query
stmt = select(Post).options(joinedload(Post.author))
posts = db.execute(stmt).scalars().unique().all()
```

3. Métodos de Eager Loading

`joinedload()` - Para relaciones 1:N (lado "uno")

```

from sqlalchemy.orm import joinedload

# Cargar post con su autor (1 query con JOIN)
stmt = (
    select(Post)
    .options(joinedload(Post.author))
    .where(Post.id == post_id)
)
post = db.execute(stmt).scalar_one_or_none()

# SQL generado:
# SELECT posts.*, authors.*
# FROM posts
# LEFT JOIN authors ON posts.author_id = authors.id
# WHERE posts.id = ?

```

selectinload() - Para relaciones 1:N (lado "muchos") y N:M

```

from sqlalchemy.orm import selectinload

# Cargar autor con sus posts (2 queries optimizadas)
stmt = (
    select(Author)
    .options(selectinload(Author.posts))
    .where(Author.id == author_id)
)
author = db.execute(stmt).scalar_one_or_none()

# SQL generado:
# Query 1: SELECT * FROM authors WHERE id = ?
# Query 2: SELECT * FROM posts WHERE author_id IN (?)

```

Combinando ambos

```

# Cargar posts con autor Y tags
stmt = (
    select(Post)
    .options(
        joinedload(Post.author),      # JOIN para autor
        selectinload(Post.tags)      # SELECT IN para tags
    )
)
posts = db.execute(stmt).scalars().unique().all()

```

4. JOINS Explícitos para Filtrar

Cuando quieres **filtrar** por campos de la relación:

```

from sqlalchemy import select

# Posts de un autor específico
stmt = (
    select(Post)

```

```

        .join(Post.author) # JOIN explícito
        .where(Author.name == "John Doe")
    )
posts = db.execute(stmt).scalars().all()

# SQL:
# SELECT posts.* FROM posts
# JOIN authors ON posts.author_id = authors.id
# WHERE authors.name = 'John Doe'

```

Filtrar por relación N:M

```

# Posts con tag específico
stmt = (
    select(Post)
    .join(Post.tags) # JOIN a través de tabla asociativa
    .where(Tag.name == "python")
)
posts = db.execute(stmt).scalars().all()

```

Posts con múltiples tags (AND)

```

# Posts que tienen AMBOS tags
stmt = (
    select(Post)
    .join(Post.tags)
    .where(Tag.name.in_(["python", "fastapi"]))
    .group_by(Post.id)
    .having(func.count(Tag.id) == 2) # Debe tener los 2
)

```

5. Ordenar por Relación

```

from sqlalchemy import select, desc

# Posts ordenados por nombre del autor
stmt = (
    select(Post)
    .join(Post.author)
    .order_by(Author.name, desc(Post.created_at))
)
posts = db.execute(stmt).scalars().all()

```

6. Subqueries

Para queries más complejas:

```

from sqlalchemy import select, func

# Autores con más de 5 posts
subq = (
    select(Post.author_id, func.count(Post.id).label("post_count"))

```

```

        .group_by(Post.author_id)
        .having(func.count(Post.id) > 5)
        .subquery()
    )

    stmt = (
        select(Author)
        .join(subq, Author.id == subq.c.author_id)
    )
    prolific_authors = db.execute(stmt).scalars().all()

```

7. Ejemplos Prácticos

Listar Posts con Autor y Tags (Optimizado)

```

def list_posts_optimized(db: Session, skip: int = 0, limit: int = 10) -> list[Post]:
    """Lista posts con sus relaciones cargadas eficientemente"""
    stmt = (
        select(Post)
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
        .order_by(desc(Post.created_at))
        .offset(skip)
        .limit(limit)
    )
    return db.execute(stmt).scalars().unique().all()

```

Buscar Posts por Tag

```

def get_posts_by_tag(db: Session, tag_name: str) -> list[Post]:
    """Obtiene posts que tienen un tag específico"""
    stmt = (
        select(Post)
        .join(Post.tags)
        .where(Tag.name == tag_name)
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
        .order_by(desc(Post.created_at))
    )
    return db.execute(stmt).scalars().unique().all()

```

Estadísticas de Autor

```

def get_author_stats(db: Session, author_id: int) -> dict:
    """Obtiene estadísticas de un autor"""
    author = db.get(Author, author_id)
    if not author:
        return None

```

```

# Contar posts
post_count = db.execute(
    select(func.count(Post.id)).where(Post.author_id == author_id)
).scalar()

# Tags únicos usados
stmt = (
    select(func.count(func.distinct(Tag.id)))
    .select_from(Post)
    .join(Post.tags)
    .where(Post.author_id == author_id)
)
unique_tags = db.execute(stmt).scalar()

return {
    "author": author.name,
    "total_posts": post_count,
    "unique_tags": unique_tags
}

```

8. Resumen de Estrategias

Situación	Estrategia	Método
Cargar 1 objeto relacionado	Eager loading	<code>joinedload()</code>
Cargar lista de objetos relacionados	Eager loading	<code>selectinload()</code>
Filtrar por campo de relación	JOIN explícito	<code>.join().where()</code>
Ordenar por campo de relación	JOIN explícito	<code>.join().order_by()</code>
Contar relacionados	Subquery o func	<code>func.count()</code>

9. `.unique()` Importante

Cuando usas `joinedload()`, puedes obtener duplicados:

```

# ❌ Puede tener duplicados
posts = db.execute(stmt).scalars().all()

# ✅ Elimina duplicados
posts = db.execute(stmt).scalars().unique().all()

```

✅ Checklist

- ☐ Entiendo el problema N+1
- ☐ Sé cuándo usar `joinedload()` vs `selectinload()`
- ☐ Puedo hacer JOINS explícitos para filtrar
- ☐ Sé usar `.unique()` cuando es necesario
- ☐ Puedo ordenar por campos de relaciones